

Sprint 9: Routing, React Frameworks, and Server Components (Sinhala Study Guide)

1. Current Progress & Sprint 9 Goal

- Concept Explanation (Sinhala):** ඔයාලා මේ වෙනකොට React වල core foundations සේම හොඳට ඉගෙනගෙන ඔවරයි. Sprint 1 ඉඳන් Sprint 8 වෙනත් Components හදලා හැටි, JSX, Props, State management (useState, useReducer), Hooks (useEffect, Custom Hooks), Context, API, performance optimize කරන හැටි (useMemo, useCallback) වගේ නැමි දෙයක්ම අපි කතා කළා.

මේවර කල් ඔයාලා React වලින් හදුවේ එක screen එකක් ඇතුලේ වැඩ කරන Single Page Applications (SPAs). නැබැයි Sprint 9 වලදී අපේ app එක "multi-page-style" application එකක් බවට පත් වෙනවා. React run වෙන්නේ එකම එක app එකක් විදිනට වුණත්, URL එක වෙනස් වෙනකොට refresh වෙන්නේ නැතුව වෙනස් වෙනස් views (පිටු) පෙන්වන්න මේකෙන් පුළුවන් වෙනවා.

මම Sprint එක අවසානයේදී ඔයාලට:

- Routing කියන්නේ මොකක්ද? SPA එකකට මේක ඕනේ වෙන්නේ ඇයි?
- React Router install කරලා basic routes හදාගන්නේ කොහොමද?
- Dynamic Routes සහ nested routes හදාගන්නේ කොහොමද? useParams සහ Outlet පාවිච්චි කරන්නේ කොහොමද?
- Index routes, 404 (Not Found) routes, සහ protected routes (gatekeeper patterns) හදන්නේ කොහොමද?
- React.lazy සහ Suspense පාවිච්චි කරලා performance වැඩි කරගන්න route-level lazy loading කරන්නේ කොහොමද?
- Next.js වගේ frameworks පාවිච්චි කරන්නේ ඇයි සහ Server Components කියන්නේ මොනවාද? යන කරුණු ගැන පැහැදිලි අවබෝධයක් ලැබෙනවා.

2. What problem does routing solve?

- Concept Explanation (Sinhala):** අපි routing පාවිච්චි කරන්න කලින් මුළු React app එකම control කළේ එකම එක App.jsx එකක් ඇතුලෙන්. එනකොට user ට දැනින්ම ලැබෙන්නේ එකම screen එකයි. URL එක වෙනස් වෙන්නේ නැහැ.

නැබැයි Routing පාවිච්චි කළාම අපේ app එක වෙනස්ම විදිනකට වැඩ කරන්න ගන්නවා. URL එක අනුව React විසින් අදාළ Component එක load කරලා screen එකට පෙන්වනවා:

- `/` -> Home page එක පෙන්වන්න (`<Home />`)
- `/about` -> About page එක පෙන්වන්න (`<About />`)
- `/products` -> Products page එක පෙන්වන්න (`<Products />`)
- `/products/keyboard` -> Product detail page එක පෙන්වන්න (`<ProductDetail />`)

URL එක වෙනස් වෙද්දී මුළු page එකම reload වෙන්නේ නැතුව අදාළ component එක විතරක් ක්ෂණිකව මාරු කරන්න routing අපිට උදව් වෙනවා.

3. React Router Installation

- Concept Explanation (Sinhala):** සාමාන්‍ය Vite React app එකකට routing එකතු කරගන්න අපි react-router කියන standard library එක install කරගන්න ඕනේ. terminal එකේ මෙම command එක run කරලා ඒක කරගන්න පුළුවන්.

- Command Block:**

```
npm i react-router
```

4. The Three Main Routing Pieces

- Concept Explanation (Sinhala):** React Router වල ප්‍රධානම කොටස් තුනක් කියන්නවා:

- Browser Router :** අපේ මුළු app එකටම browser එකේ URL එක පාවිච්චි කරලා routing කරන්න කළුවත් power එක දෙන්න මෙයා. සාමාන්‍යයෙන් මුළු routes එකම wrap කරන්නේ මෙයාගෙන්.
- Routes :** අපේ app එකේ තියෙන routes rules ඔක්කොම එකතු කරලා කියාගන්න container එක. මේක හරියට list එකක් වගේ.
- Route :** URL path එක සහ අදාළ Component එක එකිනෙක සම්බන්ධ කරන (map කරන) කෙනා. "මේ path එක අවොත්, මේ component එක පෙන්වපන්" කියලා කියන්නේ Route එකෙන්.

- Code Block:**

```
import { Browser Router, Routes, Route } from "react-router";

function Home() {

  return <h1>Home Page</h1>;

}

function About() {

  return <h1>About Page</h1>;

}

export default function App() {

  return (

    <Browser Router>

    <Routes>

    <Route path="/" element={<Home />} />

    <Route path="/about" element={<About />} />

    </Routes>

    </Browser Router>

  )

};
```

5. Sprint 9 Folder Structure

- Concept Explanation (Sinhala):** ලොකු app එකක් හදද්දී file structure එක පිළිමෙලකට තියාගන්න එක ගොඩක් වටිනවා. ඒ නිසා route-level screens වෙනමම `pages/` folder එකේත්, නැවත නැවත පාවිච්චි කරන්න පුළුවන් UI කැලි `components/` folder එකේත් දාන්න පුරුදුවෙන්න.
- Structure Breakdown (Sinhala Explanation):**

- `components/` -> reusable UI කැලි (උදා: `Navbar.jsx`, `ProtectedRoute.jsx`)
- `pages/` -> මුළු route එකක්ම නියෝජනය කරන screens (උදා: `Home.jsx`, `Products.jsx`, `Dashboard.jsx`)

6. Basic Routes Setup

- Concept Explanation (Sinhala):** අපි මුලින්ම සරල පිටු කිහිපයක් හදාගෙන, ඒ ටික App.jsx එක ඇතුලට import කරලා route rules ටික ලියන හැටි බලමු.

- Code Block:**

```
// src/pages/Home.jsx

export default function Home() {

}

return <h1>Home</h1>;

// src/pages/About.jsx

export default function About() {

  return <h1>About</h1>;

}

// src/pages/Products.jsx

export default function Products() {

  return <h1>Products</h1>;

}
```

- Integrating in App.jsx:**

```
import { Browser Router, Routes, Route } from "react-router";
import Home from "../pages/Home";
import About from "../pages/About";
import Products from "../pages/Products";

export default function App() {

  return (

    <Browser Router>

    <Routes>

    <Route path="/" element={<Home />} />

    <Route path="/about" element={<About />} />

    <Route path="/products" element={<Products />} />

    </Routes>

    </Browser Router>

  );

}
```

7. Dynamic Routes

- Concept Explanation (Sinhala):** Dynamic Route එකක් කියන්නේ URL එකේ නියෙන යම් කොටසක් dynamic විදිනට වෙනස් වෙන්න පුළුවන් route එකකට. උදාහරණයක් විදිනට `/products/1`, `/products/mouse` වගේ නැම product එකකටම වෙන වෙනම route ලියන්න ගියොත් එසා වෙනවා. ඒ වෙනුවට අපි ලියන්නේ එකම එක route එකයි: `/products/:productId`.

මෙහි නියෙන `:productId` කියන්නේ variable එකක් (URL Parameter) වගේ කියා කතා කොටසක්. මේ dynamic කොටස කියවගන්න අපි `useParams` කියන hook එක පාවිච්චි කරනවා. URL එකෙන් අපිට ලැබෙන නැම parameter එකක්ම ලැබෙන්නේ String එකක් විදිනට බව මතක තියාගන්න.

- Product Detail Screen Code Block:**

```
// src/pages/ProductDetail.jsx

import { useParams } from "react-router";

export default function ProductDetail() {

  const { productId } = useParams();

  return (

    <main>

    <h1>Product Detail</h1>

    <p>Product ID: {productId}</p>

    </main>

  );

}
```

- Adding to Routes:**

```
<Route path="/products/: productId" element={<ProductDetail />} />
```

8. Nested Routes & Outlet

- Concept Explanation (Sinhala):** Nested routes කියන්නේ තවත් route එකක් ඇතුලේ තියෙන child routes වලට, උදාහරණයක් විදිනට `/courses/react` සහ `/courses/react/lessons` කියන routes සලකන්න. මේ දෙකේම පොදු layout එකක් (CourseLayout) තියෙනවා.

React Router වලදී parent route එක ඇතුලේ child routes පෙන්වන්න අපි `Outlet` කියන component එක පාවිච්චි කරනවා. `index` route එකෙන් කරන්න මගේ. ඒකට අපි එකට නරියටම URL එක match වෙද්දී display වෙන්න ඕන default component එක (CourseOverview) තෝරන එකයි.

- Course Layout Code Block (Parent):**

```
// src/pages/CourseLayout.jsx

import { Outlet, useParams } from "react-router";

export default function CourseLayout() {

  const { courseId } = useParams();

  return (

    <main>

    <h1>Course: {courseId}</h1>

    <Outlet />

    </main>

  );

}
```

- Child Components Code Block:**

```
// src/pages/CourseOverview.jsx

export default function CourseOverview() {

}

return <p>This is the course overview.</p>;

// src/pages/CourseLessons.jsx

export default function CourseLessons() {

}

return <p>These are the course lessons.</p>;
```

- Defining Nested Routes in App.jsx:**

```
<Route path="/courses/:courseId" element={<CourseLayout />} />

<Route index element={<CourseOverview />} />

<Route path="/lessons" element={<CourseLessons />} />

</Route>
```

9. 404 Route (Catch-All)

- Concept Explanation (Sinhala):** User කෙනෙක් අපේ app එකේ නැති වැරදි URL එකක් type කළොත් පෙන්වන්න 404 page එකක් තියෙන්න ඕනේ. ඒකට අපි path එකට `*` (wildcard) පාවිච්චි කරනවා. වෙන කිසිම route එකක් match නොවුණොත් අන්තිමට මේ route එක match වෙලා `NotFound` component එක පෙන්වනවා.

- NotFound Component Code Block:**

```
// src/pages/NotFound.jsx

export default function Not Found() {

}

return (

  <main>

  <h1>404</h1>

  <p>Page not found.</p>

  </main>

);

// Route Configuration:

<Route path="*" element={<NotFound />} />
```

10. Protected Route

- Concept Explanation (Sinhala):** අපේ app එකේ dashboard වගේ සමහර pages හැමෝටම බලන්න දෙන්න බැහැ. `user log වෙලා` ඉන්නවා නම් විතරක් ඒ screen එක පෙන්වන්න අපි `ProtectedRoute` එකක් හදනවා. `user log වෙලා` නැත්නම් අපි `Mops` component එක පාවිච්චි කරලා එයාට `/login` page එකට redirect කරනවා. මේක හරියට guard gate එකක් වගේ කියා කරන්නේ.

- ProtectedRoute Code Block:**

```
//src/components/ProtectedRoute.jsx

import { Navigate } from "react-router";

export default function ProtectedRoute({ isLoggedInIn, children }) {

  if (isLoggedInIn) {

    return <Navigate to="/login" replace />;

  }

}

return children;
```

- Routing Implementation:**

```
<Route

  path="/dashboard"

  element={

    <ProtectedRoute isLoggedInIn={isLoggedInIn}>

    <Dashboard />

    </ProtectedRoute>

  }

/
```

11. Lazy Loading with React.lazy() and Suspense

- Concept Explanation (Sinhala):** සාමාන්‍යයෙන් React වල components import කරද්දී app එක start වෙද්දීම සේරම components එක පාර load වෙනවා (Eager loading). නැබැයි Dashboard වගේ ලොකු පිටු user අවශ්‍ය වුණොත් විතරක් load කරගන්න (Lazy loading) අපිට පුළුවන්.

ඒකට `React.lazy()` පාවිච්චි කරනවා. නැබැයි ඒ lazy component එක backend එකෙන් තමයි ලබාදෙනවා. එකෙන් browser එකට පෙන්වන්න යමක් තියෙන්න ඕනේ. ඒකට තමයි අපි `Suspense` component එකේ `fallback` prop එක ඇතුලට loading state එකක් දෙන්නේ.

- Lazy Loading Code Block:**

```
import { lazy, Suspense } from "react";

const Dashboard lazy(() => import("../pages/Dashboard"));
```

```
// Route එක ඇතුළේ සම්පූර්ණ ආකෘතිය:

<Route

  path="/dashboard"

  element={

    <ProtectedRoute isLoggedInIn={isLoggedInIn}>

    <Suspense fallback={<p>Loading dashboard...</p>}>

    <Dashboard />

    </Suspense>

    </ProtectedRoute>

  }

/
```

12. Complete Sprint 9 Example

- Concept Explanation (Sinhala):** දැන් අපි ඉහත ඉගෙනගන්න හැම routing concept එකක්ම (basic, dynamic, nested, index, 404, protected, lazy loaded routes) එකතු කරලා හදපු සම්පූර්ණ App.jsx එක බලමු.

- Code Block:**

```
//Src/App.jsx

import { Browser Router, Routes, Route } from "react-router";

import { lazy, Suspense } from "react";

import Home from "../pages/Home";

import About from "../pages/About";

import Products from "../pages/Products";

import ProductDetail from "../pages/ProductDetail";

import Login from "../pages/Login";

import Not Found from "../pages/NotFound";

import CourseLayout from "../pages/CourseLayout";

import CourseOverview from "../pages/CourseOverview";

import CourseLessons from "../pages/CourseLessons";

import ProtectedRoute from "../components/ProtectedRoute";

const Dashboard lazy(() => import("../pages/Dashboard"));
```

```
export default function App() {

  const isLoggedInIn = false;

  return (

    <Browser Router>

    <Routes>

    <Route path="/" element={<Home />} />

    <Route path="/about" element={<About />} />

    <Route path="/products" element={<Products />} />

    <Route path="/products/:productId" element={<ProductDetail />} />

    <Route path="/courses/:courseId" element={<CourseLayout />} />

    <Route index element={<CourseOverview />} />

    <Route path="/lessons" elements={<CourseLessons />} />

    </Route>

    <Route path="/login" element={<Login />} />

    <Route

      path="/dashboard"

      element={

        <ProtectedRoute isLoggedInIn={isLoggedInIn}>

        <Suspense fallback={<p>Loading dashboard...</p>}>

        <Dashboard />

        </Suspense>

        </ProtectedRoute>

      }

    />

  );

}
```

13. React Frameworks and Next.js Readiness

- Concept Explanation (Sinhala):** React කියන්නේ UI හදන්න විතරක් උදව් වෙන library එකක්. නැබැයි ලොකු production-ready app එකක් හදද්දී අපිට routing, optimization, SEO වගේ ගොඩක් දේවල් නැතියම config කරන්න වෙනවා.

Next.js වගේ framework එකකින් ඒ හැමදේම කලින්ම සුදානම් කරලා දෙනවා. දැනට ඔයාලා Next.js වලට සෘජුවම යන්න අවශ්‍ය නැහැ. නැබැයි මේවා පාවිච්චි කරන්නේ ඇයි කියලා දැනගෙන ඉන්න එක වටිනාව:

Tool	Main Job
React	UI Components බිල්ඩ් කරන එක
React Router	Client-side routing React app එකට එකතු කරන එක
Next.js	Routing, Server rendering, Data fetching patterns, Image optimization, Backend APIs වගේ නැමද්දි සපයන සම්පූර්ණ framework එකක් වීම

14. Server Components vs Client Components

- Concept Explanation (Sinhala):** React ලෝකයේ ප්‍රධාන වශයෙන් Components වර්ග දෙකක් තියෙන්නවා:

- Client Components:** මේවා run වෙන්නේ browser එක ඇතුලේ. ඒ නිසා browser එකේ වැඩ කරන `useState`, `useEffect`, `onClick`, `localStorage`, `window` API වගේ දේවල් පාවිච්චි කරන්න පුළුවන් මේවාට විතරයි.
- Server Components:** මේවා render වෙන්නේ server එකේදී. රිට්පස්සේ HTML විදිනට browser එකට එවනවා. Server එකේ කෙටිකලීම් run වෙන නිසා data fetch කරන්න සහ UI එක ඉක්මනට render කරලා එවන්න මේවා ගොඩක් හොඳයි. නැබැයි මේවාට interactive state/effects (`useState`, `useEffect`) පාවිච්චි කරන්න බැහැ.

Next.js වගේ frameworks වලදී interactive component එකක් හදද්දී අපි file එකේ `client.js` මේ "use client"; කියලා සඳහන් කරන්න ඕනේ.

15. Debug Task

- Concept Explanation (Sinhala):** කලින් තිබුණු broken code එකේ වැරදි 3ක් තිබුණා:

- `useParam` කියලා hook එකක් නැහැ. ඒක `useParams` වෙන්න ඕනේ.
- `<Routes>` wrapper එක කෙලින්ම දාන්න බැහැ. ඒක අනිවාර්යයෙන් `<BrowserRouter>` එකක් ඇතුලේ තියෙන්න ඕනේ.
- Route එකක් ඇතුලේ component එකක් pass කරද්දී `component={...}` හෙවෙයි පාවිච්චි කරන්නේ. `element={...}` කියන prop එකයි.

- Corrected Code Block:**

```
import { Browser Router, Routes, Route, useParams } from "react-router";

function Product() {

  const {id }= useParams();

  return <h1>{id}</h1>;

}

export default function App() {

  return (

    <BrowserRouter>

    <Routes>

    <Route path="/product/:id" element={<Product />} />

    </Routes>

    </BrowserRouter>

  );

};
```

16. Shortest Mental Model

- Concept Explanation (Sinhala):** සාරාංශයක් විදිනට මේ කරුණු මතක තියාගන්න:

- Route:** URL එක component එකකට connect කරයි.
- BrowserRouter:** App එකට routing powers ලබා දෙයි.
- Routes & Route:** Routes ලැයිස්තුවක් map කරයි.
- useParams:** Dynamic URL වල තියෙන values කියවයි.
- Nested Routes & Outlet:** Parent layout එකක් ඇතුලේ child components render කරයි.
- Protected Route:** Log වෙලා නැති අයට login page එකට හරවා යවයි (Redirects).
- React.lazy & Suspense:** ලොකු pages අවශ්‍ය වූ විට පමණක් lazy-load කරයි.
- Next.js / Frameworks:** React වලට අමතරව structure එකක් සහ app-level features ලබා දෙයි.